
Printing badges at an FE event

The print partner interface

Our hub announces each badge on the event network;
your software collects it, prints it on your own design, and
reports back.

Spec version 1

17 September 2026

Forum Europe — event badge printing

Contents

1. What this interface is for	3
2. Terms	4
3. Who does what	5
4. The network	5
5. The datagram	6
6. Collecting a badge	6
7. The waiting list	13
8. Telling us how a badge went: <code>printed</code> and <code>failed</code>	13
9. Timing, cancellation and retries	15
10. Never print a badge twice	15
11. Status codes	17
12. What you may do with the data	18
13. Testing	18
14. Troubleshooting	19
15. What this interface does not offer	20
Changes	20
A. Getting started (the kit's README)	20

When a delegate checks in at an FE event, one of our desk iPads asks our print hub for that person's badge. In print partner mode the hub does not print it itself. It renders the badge, holds it, and announces it on the event network: a small UDP message on port 8632 carrying a job id and a pickup URL.

Your software fetches that URL. It gets the badge as JSON — the delegate's name, their name in their own script if we hold one, their organisation, job title and delegate type, the exact text for the QR code, and a print-ready PNG of the whole badge. You print it on your own printers, on your own stock.

The rest of this document is the detail: the datagram, the pickup, the waiting list, every field, the timing, the two routes for telling us how a badge went, and the rules that stop a delegate walking away with two badges.

1. What this interface is for

This interface exists so that you can print our delegates' badges with your own design and your own kit. In almost every case a print partner already has a badge design the client has signed off, a printing stack they trust, and stock they have bought. What they do not have is the delegate data, and that is what we supply: for each badge, the person's name, their name in their own script where we hold one, their organisation, their job title, their delegate type, the QR payload our scanners read at the door, the badge serial, and the name of the event. You lay that out however your design says, in your own fonts and colours, on your own stock, through your own drivers and colour management. None of that is ours to specify and this document does not try to.

We also send a finished picture of the badge (`image`), and optionally a finished PDF (`pdf`). Those are there for a partner who would rather not lay anything out, and as a cross-check for one who does — they are exactly what our own thermal printers produce. They are a convenience, not the expected route.

Two things matter on the day, whatever you print on:

- **The QR code.** Encode the `qr` text exactly as we send it, and check it scans. Our door scanners read it, and a code that does not scan holds a delegate up at the barrier.
- **The name.** It has to be legible at arm's length, in the script we sent it in. A steward reads it out.

There are also two routes for telling us how a badge went, `printed` and `failed` (section 8). They are available, not required: a partner that never calls them works exactly the same.

The kit

File	What it is
<code>print-partner-spec.md</code>	This document.
<code>print-partner-client.py</code>	A working reference client: it listens, collects, saves each badge, prints (your code goes in one marked function), and reports back.
<code>fake-partner-hub.py</code>	A stand-in hub that runs on your own machine and serves invented badges, so you can build and test the whole path without us.
<code>samples/</code>	One real datagram, one real badge (a test badge, no real person), the picture from it and the same badge as a PDF.

Both scripts need Python 3.10 or later and nothing else. They run on Windows and macOS.

2. Terms

These words mean one thing each throughout this document and in the reference client's output.

Datagram — the small UDP broadcast the hub sends on port 8632 to say a badge is ready. Earlier versions of this document also called it the announcement or the doorbell.

Badge — one delegate's badge: the JSON we serve, and the thing you print from it.

Job id — the hub's identifier for one *offer* of a badge, the `id` field. A retry of the same badge can come under a new job id.

Job key — the desk's identifier for the badge itself, the `jobKey` field. It survives a retry, so it is the field to dedupe on.

Collector — your software, identified by the `X-Print-Collector` value it sends. One badge is served to one collector.

Collect — `GET` a badge. The first successful `GET` takes it, and the desk treats the badge as handed over from that moment.

Waiting list — `GET /v1/prints`: the badges nobody has collected yet.

Cancelled — a badge the hub has taken back because the desk gave up waiting (see section 9). It answers `410 Gone` from then on, and must never be printed.

3. Who does what

We provide	You provide
A port on our event router, and an Ethernet cable to it	A computer on that Ethernet cable, running your software for the whole event
The hub, and its address on the day	Listening for datagrams, and polling the waiting list
One datagram per badge a desk asks for	The design, the printers, the stock, the ribbons, the drivers and the operators
The badge data: name, local-script name, organisation, job title, delegate type, QR payload, serial, event	Your own layout — or, if you would rather not, printing the picture we send
A rendered PNG of the badge, and a PDF if the event turns it on	Your own dedupe: one badge printed once (section 10)
A stand-in hub in the kit, so you can build and test without us	Testing against it before the event, and telling us your end works

4. The network

- **Use an Ethernet cable.** We give you a port on the event router. Wi-Fi drops broadcasts and degrades in a crowd, and a badge is not something that can wait for the network to recover; we will give you Wi-Fi credentials as a fallback, not as the plan.
- **Everything is local.** Nothing traverses the internet, and UDP broadcasts do not cross routers, so a different subnet will not work and a guest network with client isolation will not work.
- **The hub's address** is given to you on the day. You can also find it: the hub is the source of every datagram, and its address is in the `pickup` URL. It advertises itself over Bonjour/mDNS as `_fehub._tcp` — `dns-sd -B _fehub._tcp` on a Mac, or any Bonjour browser.
- **Ports.** HTTP on TCP **8631**, datagrams on UDP **8632**. Your firewall needs UDP 8632 inbound and TCP 8631 outbound.
 - *Windows:* Defender asks the first time Python listens — tick **Private networks** and choose **Allow access**. If you dismissed it: Windows Security → Firewall & network protection → Allow an app through firewall, then tick Python for Private. Set the event network to **Private** as well (Settings → Network & internet → Wi-Fi → the network → Private network).
 - *macOS:* if the firewall is on, macOS asks whether Python may accept incoming connections. Choose **Allow**.
- **There is no authentication** — no key, no token, no password. The event network is the boundary, and only devices we have deliberately put on it can reach the hub. The routes below answer only while we have the hub in print partner mode.

5. The datagram

For each badge the hub broadcasts one UTF-8 JSON datagram to UDP port **8632**, to `255.255.255.255` and to the local subnet's broadcast address (for example `192.168.8.255`), **three times**, roughly 200 ms apart. A hub on more than one network sends on each, so in practice you will see the same badge announced six times or more. Every repeat carries the same `id`: act on the first and ignore the rest.

A real datagram, from `samples/doorbell.json`:

```
{ "fehub": 1, "type": "print", "id": "31f3d165-c9f9-43e0-ae4e-83ce28e966c3", "hub": "FE Partner Bench", "pickup": "http://192.168.8.20:8631/v1/prints/31f3d165-c9f9-43e0-ae4e-83ce28e966c3", "time": "2026-09-17T18:08:55.069692Z", "test": true }
```

Field	Type	Meaning
fehub	number	Always <code>1</code> . Ignore a datagram without it.
type	string	Always <code>"print"</code> . Ignore any other value.
id	string (UUID)	The job id, identical in every repeat.
hub	string	The hub's name.
pickup	string (URL)	Where to collect the badge.
time	string	When the desk asked: UTC, ISO 8601 (<code>2026-09-17T18:08:55.069692Z</code>).
test	boolean	<code>true</code> for a test or sample badge.

The datagram carries nothing about the person — no name, no organisation, no code. All of that comes from the pickup URL.

Collect from the address we gave you. Treat the datagram as "there is something to fetch", not as the address to fetch it from. Two reasons: a hub with more than one network card can name a host in `pickup` that you cannot reach, and a second hub on the network would offer you badges that yours answers `404` for. Those `404`s are harmless; ignore them.

If you have no address configured, use the datagram's source address with the port and path from `pickup`. The reference client uses the address it was started with.

Datagrams get lost. That is why we ask for an Ethernet cable, and why the waiting list in section 7 exists. Poll it however reliable the broadcasts look.

6. Collecting a badge

```
GET http://<hub>:8631/v1/prints/<id>
X-Print-Collector: acme-print-1
```

Send your collector id on every request, and use the same value after a restart. It is any text up to 100 characters: a machine name, or a UUID you generate once and keep in a file, as the reference client does in `printed.json`. It is how the hub tells your own retry apart from a second machine that should not be printing.

The hub answers `200 OK` with `Content-Type: application/json; charset=utf-8` and `Cache-Control: no-store`.

The first successful `GET` collects the badge, and from that moment the desk counts it as handed over. **One badge goes to one collector.** Repeat the request with the same `X-Print-Collector` and you get the same badge back, so a dropped connection costs you nothing. A request with a different `X-Print-Collector`, or with none, gets `409 Conflict`: somebody else is printing that badge. Section 11 says what to do about it.

A real badge, from `samples/pickup.json`, with the base64 payloads cut short:

```
{
  "id": "31f3d165-c9f9-43e0-ae4e-83ce28e966c3",
  "time": "2026-09-17T18:08:55.069673Z",
  "desk": "Bench page",
  "test": true,
  "badge": {
    "name": "Desk 1",
    "nameLocal": null,
    "organisation": "Sample badge - not a real event",
    "jobTitle": null,
    "type": "TEST",
    "qr": "FE1:D:CALIBRATION",
    "serial": null,
    "registrationId": null,
    "reprint": false,
    "reason": null
  },
  "event": null,
  "design": {
    "medium": "Label102x64",
    "showName": true,
    "showOrganisation": true,
    "showJobTitle": false,
    "showType": true,
    "showQr": true,
    "showLogo": false,
    "footerText": "bench sample",
    "textColor": null,
    "whiteAreaXMm": null,
    "whiteAreaYmM": null,
    "whiteAreaWidthMm": null,
    "whiteAreaHeightMm": null,
    "qrSizeMm": null,
    "layout": {
      "areaXMm": 3,
      "areaYmM": 3,
      "areaWidthMm": 96,
      "areaHeightMm": 58,
      "authored": false,
      "elements": [
```

```

    { "kind": "Name", "xMm": 2.5, "yMm": 2.5, "widthMm": 59.38, "heightMm": 9.7875, "sizeMm":
7.25, "bold": true, "align": "Left", "maxLines": 2 },
    { "kind": "NameLocal", "xMm": 2.5, "yMm": 12.2875, "widthMm": 59.38, "heightMm": 7.3514996,
"sizeMm": 5.6549997, "bold": false, "align": "Left", "maxLines": 1 },
    { "kind": "Organisation", "xMm": 2.5, "yMm": 25.671, "widthMm": 59.38, "heightMm":
6.0319996, "sizeMm": 4.64, "bold": false, "align": "Left", "maxLines": 1 },
    { "kind": "BadgeType", "xMm": 2.5, "yMm": 49.004, "widthMm": 59.38, "heightMm": 6.496,
"sizeMm": 4.64, "bold": true, "align": "Left", "maxLines": 1 },
    { "kind": "Qr", "xMm": 64.380005, "yMm": 14.440001, "widthMm": 29.119999, "heightMm":
29.119999, "sizeMm": 29.119999, "bold": false, "align": "Left", "maxLines": 1 }
  ]
},
"logo": null
},
"size": { "widthMm": 102, "heightMm": 64 },
"image": {
  "contentType": "image/png",
  "dpi": 203,
  "widthPx": 816,
  "heightPx": 512,
  "base64": "iVBORw0KGgoAAAANSUhEUgAAZAAAAAIA..."
},
"jobKey": "31f3d165-c9f9-43e0-ae4e-83ce28e966c3",
"pdf": {
  "contentType": "application/pdf",
  "widthMm": 102,
  "heightMm": 64,
  "base64": "JVBERi0xLjQKJdPr6eEKMSAwIG9iago8..."
}
}

```

A real delegate's badge has the same shape with the fields populated — for example `"name": "Layla Example"`, `"nameLocal": "ليلى مثال"`, `"type": "Speaker"`, `"qr": "FE1:B:K7Q2M9"`, `"serial": "K7Q2M9"`, and `event` and `design.logo.set`. `fake-partner-hub.py` serves badges of exactly that shape, including a `pdf`.

How to read the payload

It is UTF-8 JSON and may contain Arabic or other non-Latin scripts, so keep it as UTF-8 when you save or forward it. Anything documented as nullable really can be `null`. Numbers may be integers (`102`) or have decimals (`7.3514996`), and every length is in millimetres. Treat unknown fields as normal: we add fields without changing the spec version, and we never remove or rename one without one.

Top level

Field	Type	Meaning
<code>id</code>	string (UUID)	The job id — the same <code>id</code> as in the datagram.
<code>time</code>	string	When the desk asked: UTC, ISO 8601. May differ from the datagram's <code>time</code> by a millisecond.
<code>desk</code>	string, nullable	Which desk asked, for example "Desk 3". Useful if you run a printer per desk.
<code>test</code>	boolean	<code>true</code> for a test or sample badge; nobody wears it.
<code>badge</code>	object	The delegate's data — the fields below.
<code>event</code>	object, nullable	<code>id</code> (UUID) and <code>name</code> of the event. <code>null</code> for a sample from a test hub.
<code>design</code>	object	Our own layout for this badge, for reference.
<code>size</code>	object	<code>widthMm</code> and <code>heightMm</code> of the stock the badge was drawn for.
<code>image</code>	object	The rendered badge as a PNG.
<code>pdf</code>	object, optional	The rendered badge as a PDF. Absent unless the event has asked us to send one.
<code>jobKey</code>	string (UUID)	The desk's id for this badge, stable across retries. Dedupe on this — section 10.

badge — the data you fill your design with

This is the part that matters if you are printing your own design. Every field is sent whether or not our design would show it, so the decision is yours.

Field	Type	What it is, and what to do with it
<code>name</code>	string, nullable	The person's name as they registered it, already cased and folded by us — print it as sent rather than upper-casing or re-ordering it. Normally one line; allow for two, and shrink rather than truncate. Long names happen: plan for about 40 characters, and let 60 fit somehow.
<code>nameLocal</code>	string, nullable	The same person's name in their own script — usually Arabic, which is right-to-left and needs a font with Arabic coverage and proper shaping. Often <code>null</code> , and you should render nothing at all in that case rather than leaving a gap. Print it under or beside the Latin name, as your design says.
<code>organisation</code>	string, nullable	Their organisation. The longest field in practice: company names of 60–80 characters are common. One line, shrunk or ellipsised — never wrapped over the QR code.
<code>jobTitle</code>	string, nullable	Their job title. Frequently <code>null</code> , and frequently long.
<code>type</code>	string, nullable	The delegate type label: "Speaker", "Delegate", "Press", "Staff" and so on, and "TEST" on a sample. This is the field most designs colour-code or band on. The set is per-event; treat it as free text and have a default for a value you have not seen.
<code>qr</code>	string, nullable	The exact text to encode in the QR code, and nothing else — no prefix, no URL wrapper, no trailing newline. Our door scanners read this. It is short ASCII (for example <code>FE1:B:K7Q2M9</code>), so a modest error-correction level is fine; print it at least 20 mm square and keep a quiet zone around it.
<code>serial</code>	string, nullable	The badge serial, which is also inside <code>qr</code> . Useful in your own logs and for answering "did we print this one?" afterwards. <code>null</code> on test and sample badges, so never key on it alone.
<code>registrationId</code>	string (UUID), nullable	Our id for the registration. <code>null</code> for tests and samples. For your logs; not something to print.
<code>reprint</code>	boolean	<code>true</code> when this badge replaces one the delegate already had.
<code>reason</code>	string, nullable	Why it is a reprint, when the desk gave a reason ("Lost badge"). Only set when <code>reprint</code> is <code>true</code> .

The event's own name is in `event.name`, and its logo, if the event has one, is in `design.logo` as `contentType` plus `base64`.

design — our layout, if you want to mirror it

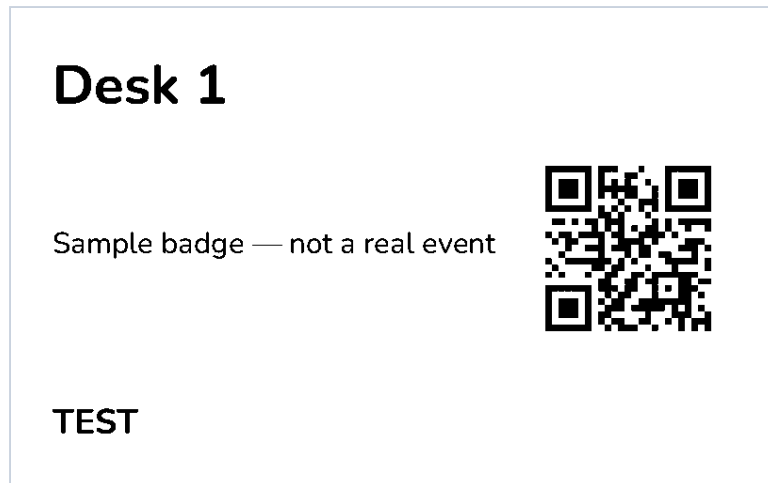
You can ignore this section entirely. It describes how we would have laid the badge out, and it is here for a partner who wants to match our house layout rather than use their own.

Field	Type	Meaning
medium	string	The stock: Label102x64 (a 102 × 64 mm label), Label89x35 (89 × 35 mm) or Preprinted4x6 (a 4 × 6 in pre-printed card).
showName, showOrganisation, showJobTitle, showType, showQr, showLogo	boolean	Whether the event's own design shows each item.
footerText	string, nullable	Fixed text along the bottom, if the event has any.
textColor	string, nullable	Text colour as #RRGGBB ; null means black.
whiteAreaXMm, whiteAreaYMm, whiteAreaWidthMm, whiteAreaHeightMm	number, nullable	On a pre-printed card, the blank window the text goes in, measured from the card's top-left corner.
qrSizeMm	number, nullable	The QR size, when the event has set one.
layout	object, nullable	Where each item sits in our rendering (below).
logo	object, nullable	The event logo: contentType (e.g. image/png) and base64 .

design.layout gives areaXMm, areaYMm, areaWidthMm, areaHeightMm — the printable area measured from the badge's top-left corner — a boolean authored (true if the event's designer positioned things themselves, false for our standard layout; either way elements is what we actually drew), and elements, one entry per item:

Field	Type	Meaning
kind	string	Name, NameLocal, Organisation, JobTitle, BadgeType (the type label) or Qr .
xMm, yMm, widthMm, heightMm	number	The item's box, measured from the top-left of the printable area , not of the badge.
sizeMm	number	Cap height of the text before any shrink-to-fit; for Qr , the side of the square.
bold	boolean	Bold text.
align	string	Left, Centre or Right .
maxLines	number	How many lines the text may take. Beyond that we shrink it, then cut it with "...".

image and pdf — the badge, already rendered



The finished picture as the hub sends it: pure black on white, one pixel per printer dot.

`image` is always present: the finished badge as a PNG, pure black on white, stored as 8-bit RGB, one pixel per printer dot at `dpi` (203 unless the event uses 300 dpi kit — at 203 dpi a 102 × 64 mm badge is 816 × 512). It is exactly what our own thermal printers burn. Print it at `dpi` with no scaling — `widthPx` / `dpi` inches wide — and it will be the right size. On a pre-printed card it contains only the variable text and the QR code, not the card's artwork.

Field	Type	Meaning
<code>contentType</code>	string	Always <code>image/png</code> .
<code>dpi</code>	number	Dots per inch, usually 203.
<code>widthPx</code> , <code>heightPx</code>	number	Pixel dimensions.
<code>base64</code>	string	The PNG, base64 (standard alphabet, padded).

`pdf` is the same badge as a one-page PDF, and is **absent unless the event has turned it on** — it is a per-event setting, off by default, that we enable when a partner asks for it. It is drawn from the same layout as the PNG, so the two are the same badge, but the text stays vector with the fonts embedded (Arabic included) and the page is the stock's size, which saves you guessing at a DPI. Take `widthMm` and `heightMm` as the real dimensions: the page box is those rounded to whole points, so it can sit a fraction of a point inside them. Print it at 100%, with no scaling and no fit-to-page. Use the PDF if your workflow prefers PDF; otherwise ignore it.

Field	Type	Meaning
<code>contentType</code>	string	Always <code>application/pdf</code> .
<code>widthMm</code> , <code>heightMm</code>	number	The page size, matching <code>size</code> .
<code>base64</code>	string	The PDF, base64.

A word on size. The PDF is roughly five times the PNG: a few hundred kilobytes against a few tens. So a badge carrying both is a few hundred kilobytes, once, at collect — and only at an event that asked for a PDF. Every other event sends the PNG alone and leaves `pdf` out. The waiting list stays small either way: it lists ids and URLs, never payloads.

7. The waiting list

```
GET http://<hub>:8631/v1/prints
```

```
{
  "waiting": [
    {
      "id": "24342316-0fe8-46f9-b509-9c83aaf88cf6",
      "jobKey": "24342316-0fe8-46f9-b509-9c83aaf88cf6",
      "time": "2026-09-17T10:37:09.690477Z",
      "test": true,
      "pickup": "http://192.168.8.20:8631/v1/prints/24342316-0fe8-46f9-b509-9c83aaf88cf6"
    }
  ]
}
```

Every badge nobody has collected, oldest first; collected and cancelled badges are not listed. The `pickup` URL is built from the host you called, so it is always one you can reach.

Poll it every 5 seconds and collect anything you do not already have. This is your safety net for a lost datagram and for the first seconds after your software starts. A desk waits only 10 seconds (section 9), so a longer poll interval loses badges.

Each entry carries `jobKey` as well as `id`, which means you can recognise a badge you have already printed and skip it **without collecting it at all**.

8. Telling us how a badge went: printed and failed

```
POST http://<hub>:8631/v1/prints/<id>/printed
POST http://<hub>:8631/v1/prints/<id>/failed
X-Print-Collector: acme-print-1
Content-Type: application/json

{"reason": "out of ribbon"}
```

These two routes are available if you want them. Call one once per badge, after your printing has returned or failed. The body is optional and only `failed` uses it: `reason` is free text, up to 200 characters, shown to our desk staff verbatim. If you cannot send the header, `{"collector": "acme-print-1"}` in the body is accepted instead.

Nothing depends on them. A client that never calls either route behaves exactly as one that does: collecting, cancellation, the waiting list and the deduplication rules are all unchanged, and no badge is held back waiting for a report. The reference client sends them by default, and you can take that out without affecting anything else it does.

What we do with them. A `failed` report reaches the desk straight away. We show it like any other print failure, with your reason in the sentence — "Desk 1: the print partner reported a failure — out of ribbon" — and we offer the operator a reprint. That reprint comes back to you as a new badge with a new `jobKey`.

This is worth having, because without it a collected badge and a printed badge look identical from the desk: the iPad says printed, and the steward turns round to hand over something that never came out of the printer. A `printed` report closes the badge quietly and interrupts nobody. Both feed counters on the hub, so we can see a printer going wrong instead of guessing from the length of the queue.

The rules, if you use them.

- Report on a badge **you** collected. The hub checks your `X-Print-Collector` against the one that took the badge; anyone else gets `409 Conflict`, as they would on a second collect, and so does a report on a badge nobody has collected.
- **The first report wins, and repeats are safe.** If your reply is lost and you send the same report again you get `200 OK` with `"applied": false` and `"state": "already reported"`. A later report that contradicts an earlier one does not change the outcome either — report once, when you know.
- **A late report is accepted.** If the desk gave up on the badge before you reported (section 9), the hub records your report, answers `200 OK` with `"applied": false` and `"state": "cancelled"`, and changes nothing: a replacement has already been printed. This is not an error and needs no retry.
- An `id` the hub has never held, or has already forgotten, answers `404` — the hub keeps a badge's record for 10 minutes from collection, which is the window a report has to land in.
- A report never re-sends a badge and never causes a reprint on its own. What it does is put the failure in front of a human at the desk, who decides.

The reply:

```
{ "id": "31f3d165-c9f9-43e0-ae4e-83ce28e966c3", "outcome": "failed", "recorded": true, "applied": true, "state": "recorded", "reportedOutcome": "failed" }
```

Field	Type	Meaning
<code>id</code>	string (UUID)	The badge you reported on.
<code>outcome</code>	string	<code>"printed"</code> or <code>"failed"</code> — the route you called.
<code>recorded</code>	boolean	Always <code>true</code> on a <code>200</code> : the report is on the badge's record.
<code>applied</code>	boolean	<code>true</code> when the report changed something. <code>false</code> for a repeat or a badge the desk had already given up on.
<code>state</code>	string	<code>"recorded"</code> , <code>"already reported"</code> or <code>"cancelled"</code> .
<code>reportedOutcome</code>	string, nullable	The outcome now standing on that badge, which for a repeat is the <i>first</i> one you sent.

If a report call itself fails, treat it the way you would a failed log write: note it locally and carry on. There is nothing to retry in a loop, and nothing about it should ever hold up the next badge.

9. Timing, cancellation and retries

- **The desk waits 10 seconds** for someone to collect a badge. Collect it in time and the desk shows it as printed; otherwise the desk shows "Not picked up by the print partner. Check their system is on this network."
- **The hub then cancels the badge**, and from that moment it answers `410 Gone`. Do not print it and do not ask for it again: the desk can already have printed a replacement, and two prints mean two badges.
- A collection and a cancellation cannot both happen. If your `GET` lands first, the badge is yours and the desk shows it as printed; if the 10 seconds ran out first, you get `410`.
- **A badge is deleted 10 minutes** after collection, or 10 minutes after it was made if nobody collected it. After that it answers `404`, and so does a report on it.
- **A hub restart cancels every uncollected badge.**

When a desk asks for the same badge again, exactly one of three things happens:

- you already collected it — the hub counts it as printed and announces nothing;
- the hub had cancelled it — it is re-announced under a **new id** with the **same jobKey**, and you should collect and print that one;
- it is still waiting — the same `id` is announced again.

So one desk request never leaves you holding two live job ids. Beyond that:

- **A reprint** — a lost or damaged badge — arrives as a genuinely new badge with `"reprint": true`, a new `jobKey`, a new `id` and usually a new `serial`, often with a `reason`. Print it: somebody is standing at the desk with nothing.
- **A new badge each day**, at events that issue one per day: new `jobKey`, new `serial`. Print each one.
- **A test badge** (`"test": true`): print it to check your kit, or just collect it. Either is correct.

10. Never print a badge twice

A delegate holding two badges is the complaint we hear at the desk, and the older one is what stops them at the door. Repeats are not a fault here — they are how the traffic normally looks. We announce each badge three times on every network, and you poll the waiting list every 5 seconds, so one badge reaches you five or six times on an ordinary morning. Printing it exactly once is as much your job as ours.

What we guarantee

- **One badge, one id** . Every datagram and every waiting-list entry for a badge carries the same `id` , and the hub never reuses an `id` for a different badge.
- **One collect**. The first `GET` takes the badge; a different collector gets `409` rather than the badge, and your own repeat `GET` gets the same bytes, so a lost reply never costs you a badge.
- **One live badge per desk request**. A desk that asks twice because it heard nothing back does not create two badges.
- **jobKey survives a retry**. A badge re-offered after a cancellation has a new `id` and the same `jobKey` ; the old `id` is already `410` .
- **410 is final** for that `id` .

What you must do

1. **Dedupe on id** — ignore a datagram for an `id` you already hold or have printed.
2. **Dedupe on jobKey** too. One `jobKey` is one badge however many `ids` it appears under, and because the waiting list carries `jobKey` you can skip a badge you have printed without collecting it.
3. **Keep that record on disk and read it at start-up**. A restart, a crash or a second copy of your software must not reprint this morning's badges. The reference client keeps `printed.json` beside the badge folder, holding every `id` and `jobKey` it has finished.
4. **Write the record before you send anything to a printer, not after**. If your software dies between collecting and printing, the badge goes unprinted — which is the failure you want, because a human can see it and fix it, whereas a duplicate walks out of the building.
5. **Send X-Print-Collector** , the same value every time, kept across restarts.
6. **Run one collector**. One computer, one process, one queue per badge. A spare machine left running, or yesterday's process nobody stopped, is the usual explanation for everything printing twice. If you must keep a spare running, give it its own `X-Print-Collector` so the hub refuses it with `409` . That is a safety net, not a design.

The fields to key on

Field	Where it appears	Use it for
<code>id</code>	datagram, waiting list, badge	This offer. Ignore repeats.
<code>jobKey</code>	waiting list, badge	The badge itself. One <code>jobKey</code> is one printed badge, permanently.
<code>badge.serial</code>	badge	The serial in the QR code — good for your logs and for after-the-fact questions. <code>null</code> on test and sample badges, so never key on it alone.

If your printer jams after you collected the badge

We have already counted the badge as collected and the desk has moved on. There is no way to hand it back. So:

1. Clear the jam and print from the copy you saved — this is why the reference client writes the PNG and JSON to disk before printing.
2. If you cannot print your copy, the desk has to print a replacement. A `failed` report (section 8) tells them without anybody walking over; so does telling the staff. Their reprint arrives as a new badge with a new `jobKey`, and the delegate ends up with one badge.

Never retry by collecting again. The `id` you hold is the only copy we keep, and collecting somebody else's badge by mistake is exactly the duplicate we are all trying to avoid.

Test this on purpose. `fake-partner-hub.py` tries to make your client print twice. It announces everything three times, re-offers one badge under a new `id` with the same `jobKey` after you collect it ("Robin Retry"), and leaves another on the waiting list for ever after collection ("Ash Repeat"). Run two copies of your client at once, restart them, and check that each badge came out exactly once.

11. Status codes

Errors are `application/problem+json` with a plain-English `detail`:

```
{"type":"https://tools.ietf.org/html/rfc9110#section-15.5.11","title":"Print cancelled","status":410,"detail":"This print was cancelled because it wasn't collected in time."}
```

Status	What it means	What to do
200	The badge (collect), the list (waiting list), or your report was taken.	Print it; collect what you do not have; carry on.
404	No badge with that <code>id</code> on this hub — it was never here, or its 10 minutes are up.	Drop it. On a report, nothing more to do.
409	Two cases, told apart by <code>title</code> . " Not in print partner mode ": the hub is driving its own printers. " Already collected " / " Not your badge ": another collector holds that badge.	Not in partner mode: keep polling every 5 s and tell us if it persists. Another collector: do not print it , and find the other collector (section 10).
410	The hub cancelled the badge because nobody collected it in time.	Do not print it. Drop it and never ask again.
503	The hub is locked because nobody has started the desk yet today.	Keep polling every 5 s.

If you cannot reach the hub at all, check you are on the event network, then retry with a growing delay. The reference client starts at 5 s and doubles to a maximum of 60 s.

12. What you may do with the data

- You receive only what is printed on a badge, and only for badges a desk has asked for. There is no attendee list, no search, and no check-in data.
- Keep badge data only as long as you need it to print. Delete the saved badges — the reference client's PNG, PDF and JSON files — at the end of the event.
- Do not send it anywhere else, and do not use it for anything but printing.

13. Testing

On your own machine, against the stand-in hub. In one window:

```
python fake-partner-hub.py (Windows: py -3 fake-partner-hub.py)
```

In another:

```
python print-partner-client.py --hub 127.0.0.1
```

The client saves three waiting badges straight away, finds a fourth only through the waiting list, and refuses a fifth that has been cancelled — which is the correct behaviour.

For more badges, open `http://127.0.0.1:8631/` and use **Send a sample print**. That page tells you whether you collected the badge within 10 seconds, and shows the reports it has had from you. The stand-in hub also takes `--every 20` (a sample badge every 20 seconds), `--mode manage` (answers 409) and `--mode locked` (answers 503).

One test worth doing deliberately: stop your client, send a sample badge, wait 10 seconds, start the client again, and confirm that badge does not print.

Testing the PDF path. The stand-in hub sends a pdf on every badge, so you can try that route before you have seen our hardware. Start it, let your client collect a badge, and print the saved `<id>.pdf` on the stock you intend to use. Start it with `--no-pdf` and it behaves like an event that has the setting switched off, which is worth checking your code takes in its stride. The kit also carries `samples/pickup.pdf`, a badge our own hub produced, if you would rather print one before you write anything.

There is no joint test before the event. The first time your software meets our hub is at the venue on setup day. That is why the stand-in hub is in the kit. It sends the same datagrams and serves the same badges as the real one, on the same ports, with the same status codes, so a system that works against it works against ours. Build and test against it until you are satisfied, then tell us you are.

What to have ready on setup day:

- the laptop or machine that will run all event, with your software installed and already working end to end against the stand-in hub;
- your printers, your stock and enough consumables for the day;
- an Ethernet cable for our router — we provide the port;
- somebody who can change your settings on the spot, rather than a machine nobody present can reconfigure.

On the day we put the hub into print partner mode from our app and send you a test badge from it. Our hub also serves a page at `http://<hub>:8631/` with **Use print partner mode**, **Send a sample print**, the time of the last collection, and the number of badges waiting.

14. Troubleshooting

Symptom	Likely cause
Nothing arrives and you cannot reach the waiting list	You are not on the event network, the hub address is wrong, or something is blocking TCP 8631.
The waiting list works but no datagrams arrive	A firewall is blocking UDP 8632 inbound (on Windows: the network is Public, or Python is not allowed), or you are on Wi-Fi that isolates clients. Use the Ethernet cable. Badges still arrive through the waiting list, just later.
"cannot listen on UDP 8632"	Another program holds the port exclusively. Close it.
Constant 409 , "Not in print partner mode"	The hub is not in partner mode. Ask us to switch it.
409 , "Already collected" or "Not your badge"	Another collector has that badge: a spare machine, an old process, or a second copy of your software. Stop the extra one; do not print the badge.
A delegate has two badges	Two collectors, or a client with no durable record of what it printed. Read section 10.
503	Nobody has started the desk yet. It clears when they do.
The desk says "Not picked up by the print partner"	You did not collect within 10 s. Check your software is running and on the network.
410	The hub cancelled the badge. Refusing to print it is the correct behaviour.
An Arabic name looks wrong in a saved file	Save as UTF-8. Your console may render it badly even when the file is correct.

15. What this interface does not offer

- No attendee list, no search, no check-in data.
- No printer telemetry: queue depth, ink levels and the state of your kit are yours. The `printed` and `failed` routes in section 8 are per badge and optional, and there is nothing else pointing back at us.
- No other part of the hub.
- Nothing over the internet.

Changes

- 1 (17 Sep 2026): the first version. It covers the datagram, the pickup, the waiting list, every field and status code, the optional `pdf` alongside `image`, the two optional routes for telling us how a badge went (`printed` and `failed`), the rule that one badge is printed once, and how to test against the stand-in hub before setup day.

A. Getting started (the kit's README)

The developer kit ships this as `README.txt`. It is the short way in; the sections above are the reference.

FE PRINT PARTNER KIT

=====

This kit is for a print supplier printing the name badges at an FE event.

How it works. Each time a check-in desk asks for a badge, our print hub broadcasts a small UDP message on port 8632. Your computer then fetches that badge from the hub over HTTP on port 8631. You get the delegate's data as JSON, a rendered PNG of the badge, and a PDF if the event has asked for one. You print it on your own kit. There is no login and no key: being on the event network is what gets you in.

WHAT THIS IS FOR

Most partners print THEIR OWN badge design and take only our data to fill it in: the delegate's name, their name in their own script where we have one, their organisation, job title and delegate type, the exact QR text, the badge serial and the event name. Your layout, stock, fonts, colours, drivers and colour management stay yours. None of that is ours to specify.

If you would rather not lay anything out, print the PNG we send. It is exactly what our own thermal printers produce. The optional PDF is the same badge as vector, at the stock's exact size in millimetres. Use whichever suits you.

Two things make the badge work on the day, whatever you print on:

- the QR text, encoded exactly as sent and scannable. Our door scanners read it, and a code that does not scan holds a delegate up.
- the name, legible at arm's length and in the script we sent it in.

There are also two routes for telling us how a badge went, "printed" and "failed". They are available, not required - see TELLING US HOW IT WENT.

WHAT WE GIVE YOU

The event network and the hub
The UDP message on 8632
The badge over HTTP on 8631:

WHAT YOU BUILD

Your own printing, from the badge
data or from our picture
Your queue, retries and operators

the data, the QR text, our | The printer, the stock and the
design, a PNG, sometimes a PDF | settings

print-partner-spec.md is the full interface: the datagram, the pickup, the waiting list, every field, every status code and the reports. That file is the reference; this one only gets you started.

TERMS

datagram	The UDP message on port 8632 announcing that a badge is ready. Older documents called it the announcement or the doorbell.
badge	One delegate's badge: the JSON, and the thing you print.
job id	The hub's id for one offer of a badge ("id").
job key	The desk's id for the badge itself ("jobKey"), stable across a retry.
collector	Your software, identified by X-Print-Collector.
collect	GET a badge. The first successful GET takes it.
waiting list	The badges nobody has collected yet.
cancelled	A badge the hub took back. It answers 410. Never print it.

REQUIREMENTS

Python 3.10 or later, and nothing else.

macOS	Install from https://www.python.org/downloads/ , or with Homebrew: <code>brew install python</code>
Windows	Install from https://www.python.org/downloads/ and tick "Add python.exe to PATH" on the first screen. The bundled "py" launcher works too.

Check it: open Terminal (macOS) or Command Prompt (Windows) and run
`python3 --version` (macOS)
`py -3 --version` (Windows)

QUICK START A – one computer, no event

The kit contains a stand-in hub that serves invented badges, so you can build against the whole path without our hardware.

- macOS
1. Double-click `start-fake-hub-mac.command`. Leave the window open.
 2. Double-click `start-client-mac.command`. Press Enter to accept 127.0.0.1 as the hub address.

- Windows
1. Double-click `start-fake-hub-windows.bat`. Leave the window open.
 2. Double-click `start-client-windows.bat`. Press Enter to accept 127.0.0.1 as the hub address.

The stand-in hub sends a PDF on every badge, so you can try that route too: print a saved `<id>.pdf` on the stock you mean to use. Start it with `--no-pdf` and it behaves like an event that has the setting switched off, which is worth checking your code takes in its stride.

Within a few seconds the client saves each badge into the "badges" folder beside these files: `<id>.png` (the rendered picture), `<id>.json` (everything, including the data you would fill your own design with) and `<id>.pdf` when the badge carries one.

What you should see, and why:

- one badge takes about 10 seconds, because it arrives only through the client's regular poll of the waiting list;
- one badge is cancelled, and the client skips it. That is correct.

- two badges are deliberate attempts to make you print the same badge twice. The client does not, and neither must your system. Read NEVER PRINT A BADGE TWICE below.

If macOS refuses to open the .command file, right-click it and choose Open, or run it from Terminal: `bash start-client-mac.command`

Stop either window with Ctrl-C, or close it.

QUICK START B - at the event, against the real hub

1. Put an Ethernet cable into our router; we give you the port. Use the cable. Wi-Fi drops broadcasts and degrades in a crowd, so we give you Wi-Fi details only as a fallback.
2. We give you the hub's address on the day (for example 192.168.8.20).
3. Start the client and give it that address:
macOS double-click start-client-mac.command
Windows double-click start-client-windows.bat
or directly:
 `bash start-client-mac.command 192.168.8.20`
 `start-client-windows.bat 192.168.8.20`
4. Ask us to send a test badge. A test badge has "test": true.

There is no joint test before the event: setup day at the venue is the first time your software meets our hub. Test against the stand-in hub until then - it sends the same datagrams and the same badge payloads on the same ports, so a system that works against it works against ours.

Have ready on setup day: the machine that will run all event, with your software installed and already working against the stand-in hub; your printers, stock and consumables; an Ethernet cable for our router; and somebody who can change your settings on the spot.

Do not run the stand-in hub at the event.

WHAT EACH FILE IS

README.txt	This file.
print-partner-spec.md	The full interface: the datagram, the pickup, the waiting list, the fields, the reports and the status codes.
print-partner-client.py	A working reference client. Your printing goes in the function <code>print_badge()</code> ; everything else can stay as it is.
fake-partner-hub.py	The stand-in hub, serving invented badges. For development only.
start-client-mac.command	Starts the client (macOS).
start-fake-hub-mac.command	Starts the stand-in hub (macOS).
start-client-windows.bat	Starts the client (Windows).
start-fake-hub-windows.bat	Starts the stand-in hub (Windows).
samples/	One real datagram, one real badge, its picture and the same badge as a PDF (pickup.pdf) - print that on your stock before you write anything.
badges.printed.json	The client's record of what it has printed. Do not delete it while the event is running.

The launchers take optional arguments:

```
start-client-* [hub address] [hub port] (defaults 127.0.0.1, 8631)
start-fake-hub-* [port] (default 8631)
Windows only: add /nopause to close the window without a key press.
```

To run the client directly:

```
python3 print-partner-client.py --hub <address> [--port 8631] [--out badges]
                                [--record FILE] [--forget]
```

TELLING US HOW IT WENT

If you want to, you can tell the hub what happened to a badge:

```
POST http://<hub>:8631/v1/prints/<id>/printed
POST http://<hub>:8631/v1/prints/<id>/failed {"reason": "out of ribbon"}
```

with your X-Print-Collector header on the request. The reference client does this already: "printed" when its print hook returns, "failed" with the error text when it raises.

What we do with them. A "failed" report reaches the desk and is shown like any other print failure, with your reason in the sentence ("Desk 1: the print partner reported a failure – out of ribbon"). We offer the operator a reprint, which reaches you as a new badge with a new job key.

Without it, a jam and a successful print look identical from the desk: the iPad says printed, the steward turns round, and there is nothing to hand over. A "printed" report closes the badge quietly.

Nothing depends on them. A client that never sends them behaves exactly the same, and no badge waits for one. Report once per badge; a repeat is accepted and changes nothing. If a report call fails, note it and carry on – there is nothing to retry in a loop, and it should never hold up the next badge. Section 8 of the spec has the rules and the status codes.

NEVER PRINT A BADGE TWICE

A delegate holding two badges is the complaint we hear at the desk. The hub announces each badge three times on every network, and the waiting list repeats it every 5 seconds, so you will hear about one badge five or six times on an ordinary morning. That is normal, not a fault – your software has to print it once anyway.

Each badge carries a job id ("id") and the desk's job key ("jobKey"). The hub serves a badge to ONE collector and answers 409 to a second. The reference client records every job id and job key it has printed in badges.printed.json, checks that file before it collects and again before it prints, and sends its own X-Print-Collector on every request, so a restart, a repeat datagram or a second client never reprints a badge. Copy that behaviour.

Run ONE collector. A spare machine, or yesterday's process nobody stopped, is the usual explanation for everything printing twice.

A reprint, or a badge for a new day, arrives with a NEW job key. It is a new badge and you must print it.

--forget empties the record so you can reprint a test badge. Never use it at an event.

Section 10 of print-partner-spec.md has the detail.

FIREWALL

The client must receive UDP on port 8632 and connect out to TCP 8631. Your firewall has to allow both.

- | | |
|---------|--|
| macOS | If macOS asks whether Python may accept incoming network connections, choose Allow. If you missed it: System Settings > Network > Firewall > Options, and set Python to "Allow incoming connections". |
| Windows | When Defender asks about Python, tick "Private networks" and click Allow. If you missed it: Control Panel > Windows Defender Firewall > Allow an app through Windows Defender Firewall, and tick Private for |

Python. Set the event network as Private as well.

RULES THAT MATTER

- A badge that answers 410 is cancelled. Do not print it: the desk stopped waiting and may already have printed a replacement.
- The hub can announce the same badge more than once. Print it once.
- Print the QR text exactly as sent.
- A badge holds personal data. Delete the saved files after the event.

TROUBLESHOOTING

No badges arrive:

- Is your Ethernet cable in our router, and is the link light on?
- Is the firewall blocking Python? See FIREWALL above.
- Start the client with the hub's address (`--hub <address>`, or type it when the client asks). It then also polls the hub's waiting list every 5 seconds, which works even when the network swallows the datagrams.
- "cannot reach the hub": check the address with us, and check you can open `http://<address>:8631/v1/prints` in a browser.
- "hub not ready (HTTP 409)": the hub is not in print partner mode yet. Ask us to switch it.
- "hub not ready (HTTP 503)": nobody has started the desk. Wait, or ask us.

"Address already in use" with the stand-in hub:

A second copy is already running. Close it, or start on another port:
`start-fake-hub-mac.command 8641` and then `start-client-mac.command 127.0.0.1 8641` (the same on Windows).

Only one program per machine can listen for the datagrams at a time. If something else holds the port, the client still finds badges through the waiting list.